Praktikum JavaScript & TypeScript

Modul 2 - Perulangan dan Fungsi

1. Tujuan Praktikum

Setelah mengikuti praktikum ini, peserta mampu:

- Menyimpan banyak data dalam array sederhana
- Memilih `for`, `while`, atau `for...of` sesuai masalah
- Menghentikan perulangan lebih awal dengan `break` atau `return`
- Menulis fungsi dengan tiga gaya: declaration, expression, dan arrow
- Memisahkan logika perhitungan dari mencetak output
- Mengirim fungsi sebagai argumen ke fungsi lain

2. Dasar Teori

2.1. Array Sederhana

Perulangan hampir selalu dipakai untuk memproses banyak data. Di JavaScript, banyak data disimpan dalam **array**. Di modul ini kita cukup memakai empat hal dasar. Method array lain dibahas di Modul 3.

```
const tagihan = [150000, 75000, 300000];
console.log(tagihan[0]);      // 150000  (index mulai dari 0)
console.log(tagihan.length);  // 3
tagihan.push(50000);          // tambah di akhir
console.log(tagihan[3]);      // 50000
console.log(tagihan[10]);     // undefined (bukan error)
```

Listing 1: array dasar

> **⚠ Perhatian**
>
> Berbeda dengan C, membaca index di luar batas **tidak** error. Hasilnya `undefined`. Jika `undefined` ikut dijumlahkan, hasilnya `NaN` (*Not a Number*), dan program tetap berjalan dengan hasil salah.

2.2. for klasik

Bentuknya sama dengan C: inisialisasi, kondisi, lalu langkah. Pakai `for` klasik jika kamu butuh **index**, misalnya untuk tahu posisi data.

```
const antrean = ["U", "A", "G"];
for (let i = 0; i < antrean.length; i++) {
  console.log(`Nomor ${i + 1}: poli ${antrean[i]}`);
}
```

Listing 2: for klasik

- `break` menghentikan perulangan.
- `continue` melompati sisa isi perulangan dan lanjut ke putaran berikutnya.
- Di dalam fungsi, `return` menghentikan perulangan **dan** fungsinya sekaligus.

> **⚠ Perhatian**
>
> Kesalahan paling umum adalah *off-by-one*. Index terakhir array adalah `length - 1`, jadi kondisinya `i < arr.length`, bukan `i <= arr.length`.

2.3. while

Pakai `while` jika jumlah putaran **belum diketahui** di awal, misalnya "berapa bulan sampai tabungan mencapai target".

```
let saldo = 0;
let bulan = 0;
while (saldo < 1000000) {
  saldo += 300000;
  bulan++;
}
console.log(bulan); // 4
```

Listing 3: while

> **⚠ Perhatian**
>
> Pastikan kondisi `while` suatu saat bernilai `false`. Jika setoran di contoh atas bernilai `0`, perulangan tidak pernah berhenti (*infinite loop*) dan program macet. Tangani kasus ini dengan guard clause **sebelum** masuk `while`.

Ada juga `do...while`, yang menjalankan isi perulangan minimal sekali sebelum mengecek kondisi. Bentuk ini jarang dipakai.

2.4. for…of dan for…in

`for...of` mengambil **nilai** setiap elemen satu per satu, tanpa index. Pakai bentuk ini jika kamu hanya butuh nilainya. Tidak ada index berarti tidak ada risiko off-by-one.

```
const mutasi = [500000, -150000, 75000];
let total = 0;
for (const m of mutasi) {
  total += m;
}
console.log(total); // 425000
```

Listing 4: for...of

Perhatikan bahwa `m` boleh `const`, karena setiap putaran membuat variabel `m` baru.

> **⚠ Perhatian**
>
> `for...in` mengambil **key** (nama properti), bukan nilai. Pada array, key berupa string `"0"`, `"1"`, dan seterusnya. Jangan pakai `for...in` untuk array. `for...in` dipakai untuk object (Modul 4).

> **✎ Catatan**
>
> Pertanyaan interview yang sering muncul: "kapan pakai `for` dan kapan pakai `forEach`?". `forEach` dibahas di Modul 3. Jawaban singkatnya: `for` dan `for...of` bisa dihentikan dengan `break`, sedangkan `forEach` tidak bisa.

2.5. Fungsi

2.5.1. Tiga cara menulis fungsi

```
// 1. function declaration
function luas(p, l) {
  return p * l;
}

// 2. function expression
const keliling = function (p, l) {
  return 2 * (p + l);
};

// 3. arrow function
const diagonalKuadrat = (p, l) => p * p + l * l;
```

Listing 5: declaration, expression, arrow

Arrow function punya dua bentuk:

- **Tanpa kurung kurawal**: hasil ekspresi otomatis dikembalikan (*implicit return*).
- **Dengan kurung kurawal**: wajib menulis `return`. Tanpa `return`, fungsi mengembalikan `undefined`.

```
const ppnA = (harga) => harga * 11 / 100;            // 11000
const ppnB = (harga) => { return harga * 11 / 100; }; // 11000
```

Listing 6: dua bentuk arrow function

> **⚠ Perhatian**
>
> Function declaration bisa dipanggil sebelum baris definisinya (*hoisting*). Function expression dan arrow function yang disimpan di `const` **tidak** bisa: memanggilnya sebelum definisi menghasilkan `ReferenceError`.

2.5.2. Parameter default dan return

```
function hitungAdmin(nominal, biaya = 2500) {
  return nominal + biaya;
}
console.log(hitungAdmin(100000));       // 102500
console.log(hitungAdmin(100000, 0));    // 100000
```

Listing 7: parameter default

Fungsi tanpa `return` mengembalikan `undefined`. Parameter yang tidak diisi saat pemanggilan juga bernilai `undefined`.

2.5.3. Hitung di fungsi, cetak di luar

Fungsi yang **mengembalikan** nilai lebih berguna daripada fungsi yang langsung `console.log`. Nilai yang dikembalikan bisa dicetak, dijumlahkan, dikirim ke server, atau dites. Nilai yang langsung dicetak hanya bisa dilihat.

```
// kurang baik: hanya bisa dicetak
function cetakTotal(a, b) { console.log(a + b); }

// lebih baik: pemanggil bebas memakai hasilnya
function total(a, b) { return a + b; }
console.log(`Total: ${total(2, 3)}`);
```

Listing 8: pisahkan perhitungan dan output

2.5.4. Fungsi adalah nilai

Di JavaScript, fungsi bisa disimpan di variabel dan dikirim sebagai argumen ke fungsi lain. Fungsi yang dikirim ini disebut *callback*. Konsep ini dipakai di semua method array di Modul 3.

```
const tambahPajak = (x) => x + x * 10 / 100;
const tambahOngkir = (x) => x + 15000;

function hitungAkhir(harga, aturan) {
  return aturan(harga);
}

console.log(hitungAkhir(100000, tambahPajak));  // 110000
console.log(hitungAkhir(100000, tambahOngkir)); // 115000
```

Listing 9: fungsi sebagai argumen

Perhatikan: yang dikirim adalah `tambahPajak`, bukan `tambahPajak()`. Tanpa tanda kurung, kita mengirim fungsinya. Dengan tanda kurung, kita mengirim **hasil** pemanggilannya.

Coba Sendiri

Buat file `Jawaban/coba2.js`. Tulis tebakanmu sebagai komentar di ujung setiap baris sebelum menjalankan, lalu bandingkan.

```
const nilai = [10, 20, 30];
for (const i in nilai) console.log(i, typeof i);
for (const v of nilai) console.log(v, typeof v);

let jumlah = 0;
for (let i = 0; i <= nilai.length; i++) jumlah += nilai[i];
console.log(jumlah);

const kali2 = (x) => { x * 2 };
console.log(kali2(5));

console.log(sapa("Dion"));
function sapa(nama) { return `Halo ${nama}`; }
```

1. Tebakan mana yang salah? Jelaskan penyebabnya.
2. Perbaiki perulangan `jumlah` dan fungsi `kali2` agar hasilnya `60` dan `10`.
3. Ubah `sapa` menjadi arrow function yang disimpan di `const`, tanpa memindahkan barisnya. Apa yang terjadi, dan mengapa?

Jawab pertanyaan di atas sebagai komentar di bagian bawah `coba2.js`.